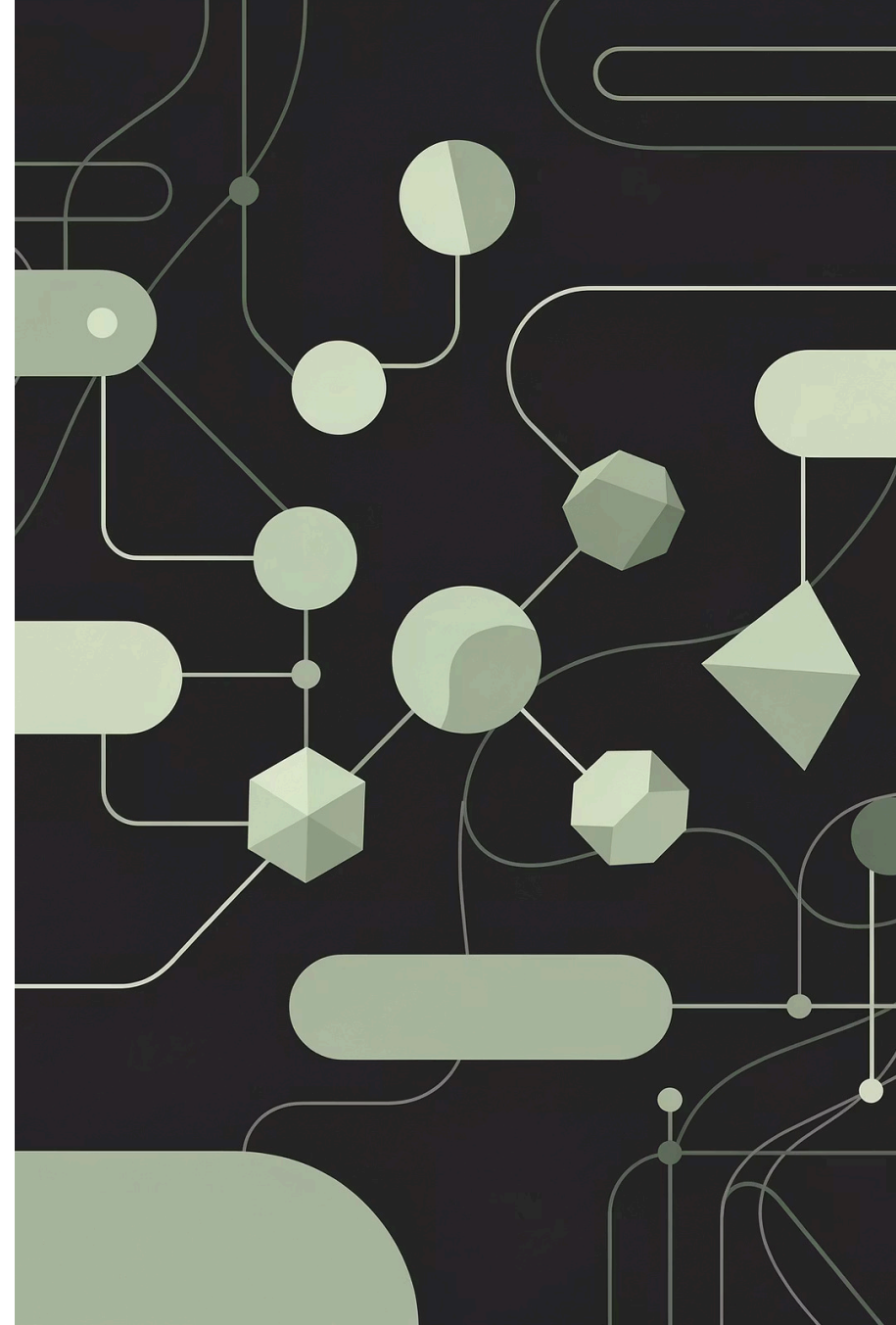


UNLOX CHAPTER 3.1

An Overview of Key Data Structures

Specialized formats for organizing, storing, and manipulating data — the essential building blocks every Python developer needs for AI and data science.



What Are Data Structures?

Data structures are **specialized formats** for organizing and storing data in a computer so that it can be accessed and modified efficiently. Think of them as different types of containers — each designed for a specific purpose.

Python provides several powerful **built-in data structures** that form the backbone of nearly every AI and data science workflow. Choosing the right data structure can mean the difference between code that runs in seconds and code that takes hours.

In this chapter, we'll explore five essential structures that every Python programmer must master.



UNLOCK

AT A GLANCE

The Five Essential Structures



Lists

Ordered & mutable



Tuples

Ordered & immutable



Dictionaries

Key-value pairs



Sets

Unique elements



Strings

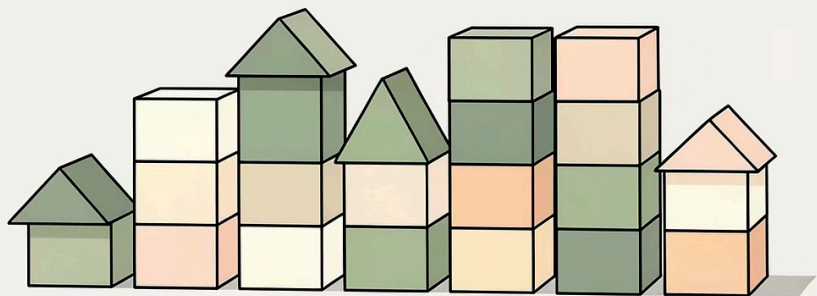
Character sequences

Each structure has distinct properties that make it ideal for specific tasks in AI and data science. Understanding **when** to use each one is just as important as knowing **how** to use it.

UNLOCK

LISTS

Lists — Ordered & Mutable Collections



Lists are Python's most versatile data structure. They are **ordered**, meaning elements maintain their position, and **mutable**, meaning you can add, remove, or modify items after creation.

Mixed Data Types

A single list can hold integers, strings, floats, and even other lists — all at once.

Dynamic Sizing

Elements can be appended, inserted, or removed at any time using built-in methods like `.append()` and `.pop()`.

AI Use Case

Lists are perfect for storing sequences of data points, feature vectors, or batches of training samples before conversion to arrays.

Lists in Action

Creating & Modifying

```
# Create a list of sensor readings
readings = [23.5, 24.1, 22.8, 25.0]

# Add a new reading
readings.append(24.7)

# Modify the second element
readings[1] = 24.3

# Remove the last element
readings.pop()

print(readings)
# [23.5, 24.3, 22.8, 25.0]
```

Key Methods to Know

- `.append(x)` — Add item to the end
- `.insert(i, x)` — Insert item at index `i`
- `.pop(i)` — Remove and return item at index
- `.sort()` — Sort the list in place
- `.reverse()` — Reverse the list order
- `len(list)` — Get the number of elements



Lists use zero-based indexing. The first element is at index `0`, not `1`.

Tuples — Ordered & Immutable Collections

Tuples look similar to lists but come with one critical difference: once created, their elements **cannot be changed**. This immutability makes them ideal for data that should remain constant throughout your program's execution.

Immutable by Design

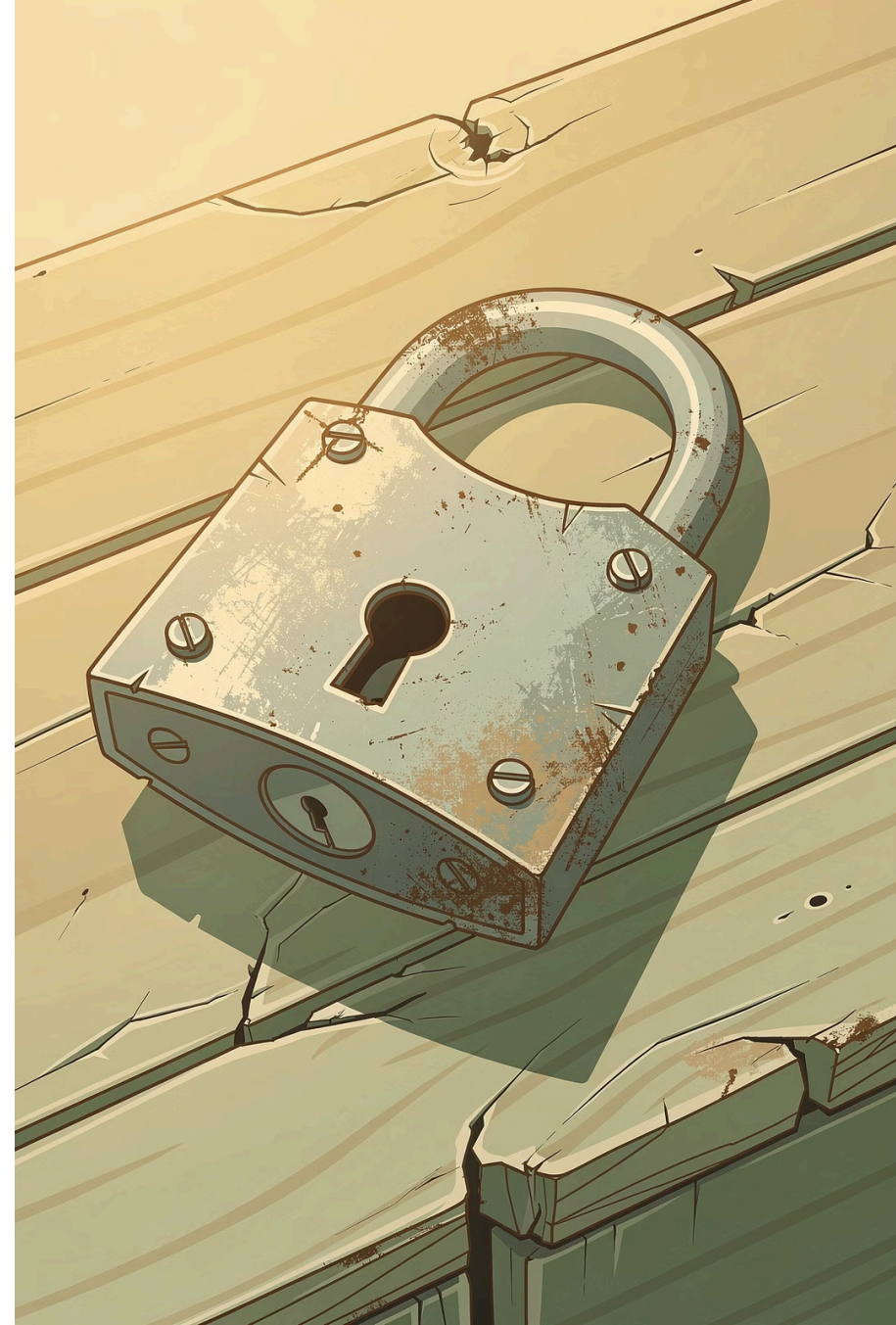
Elements cannot be added, removed, or modified after creation — protecting data integrity.

Faster Performance

Immutability allows Python to optimize memory allocation, making tuples faster than lists for read operations.

Hashable

Unlike lists, tuples can be used as dictionary keys or set elements — a powerful advantage.



When to Choose Tuples Over Lists

Code Example

```
# Fixed model hyperparameters
config = (0.001, 64, 100)
learning_rate, batch_size, epochs = config

# Coordinates that shouldn't change
origin = (0, 0)

# Tuples as dictionary keys
pixel_colors = {
    (0, 0): "red",
    (1, 0): "blue",
    (0, 1): "green"
}
```



Use a Tuple When...

The data represents a fixed record (e.g., RGB values, GPS coordinates, database rows) or when you need a hashable, unchangeable sequence.

Use a List When...

You need to dynamically add, remove, or sort elements during runtime — such as accumulating predictions or building datasets iteratively.

Dictionaries — Key-Value Pairs

Dictionaries store data as **key-value pairs**, enabling lightning-fast lookups by key. They are the go-to structure for representing structured, labeled data — similar to a real-world dictionary where you look up a word (key) to find its definition (value).

Core Characteristics

- **Keys must be unique** and immutable (strings, numbers, or tuples)
- **Values can be anything** — lists, other dictionaries, objects
- **$O(1)$ average lookup time** — extremely fast regardless of size
- Maintains **insertion order** since Python 3.7+

Code Example

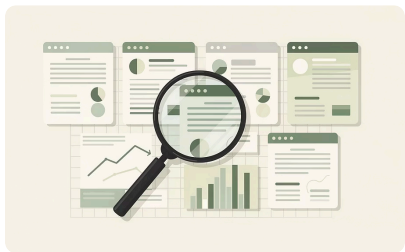
```
# Store model metadata
model = {
    "name": "RandomForest",
    "accuracy": 0.94,
    "features": ["age", "income"],
    "trained": True
}

# Access by key
print(model["accuracy"]) # 0.94

# Add a new key-value pair
model["version"] = 2.1
```

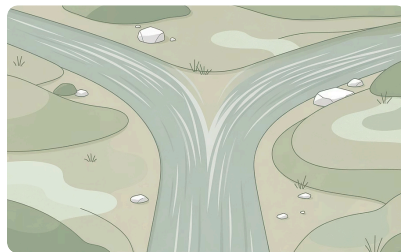


Dictionary Methods & AI Applications



Fast Lookups

Use `.get(key, default)` to safely access values without raising errors. Ideal for processing JSON responses from APIs.



Merging Data

Combine dictionaries with `.update()` or the `|` operator (Python 3.9+). Essential for merging configuration files and hyperparameters.



Nested Structures

Dictionaries can contain other dictionaries, enabling complex data representations like JSON — the standard format for AI model configs.

Sets — Unique & Unordered Collections

Sets automatically eliminate duplicates and provide **blazing-fast membership testing**. They support mathematical set operations like union, intersection, and difference — making them invaluable for data cleaning and analysis.

Code Example

```
# Remove duplicates instantly
raw_tags = ["AI", "ML", "AI", "NLP", "ML"]
unique_tags = set(raw_tags)
# {'AI', 'ML', 'NLP'}

# Set operations
train = {"img1", "img2", "img3", "img4"}
test = {"img3", "img4", "img5", "img6"}

overlap = train & test # {'img3', 'img4'}
all_imgs = train | test # Union of both
only_train = train - test # {'img1', 'img2'}
```

Why Sets Matter in Data Science

- **Deduplication** — Clean datasets by removing repeated entries in a single step
- **Fast membership checks** — `x in my_set` runs in $O(1)$ average time
- **Data validation** — Check overlap between training and test sets to prevent data leakage
- **Feature selection** — Use intersection to find common features across datasets

String Manipulation — Working with Text

Strings are **sequences of characters** and one of the most frequently used data types in AI, especially in Natural Language Processing (NLP). Python provides a rich set of built-in methods for slicing, splitting, and formatting text data.

1

Slicing

Extract portions of a string using index notation: `text[start:end]`. Negative indices count from the end.

2

Splitting

Break a string into a list of substrings with `.split(delimiter)`. Essential for tokenizing sentences into words.

3

Formatting

Build dynamic strings using f-strings: `f"Accuracy: {score:.2f}%"`. Clean, readable, and Pythonic.

String Operations in Practice

Slicing & Splitting

```
text = "Machine Learning is powerful"

# Slicing
print(text[0:7]) # "Machine"
print(text[-8:]) # "powerful"

# Splitting into words
words = text.split(" ")
# ["Machine", "Learning", "is", "powerful"]

# Join words back together
rejoined = "-".join(words)
# "Machine-Learning-is-powerful"
```

Formatting & Cleaning

```
model_name = "neural network"
accuracy = 0.9534

# f-string formatting
report = f"Model: {model_name.title()}"
score = f"Accuracy: {accuracy:.1%}"
# "Model: Neural Network"
# "Accuracy: 95.3%"

# Common cleaning operations
raw = " Hello, World! "
clean = raw.strip().lower()
# "hello, world!"
```

📄 🧹 Text cleaning — stripping whitespace, lowercasing, and removing punctuation — is the **first step** in nearly every NLP pipeline.

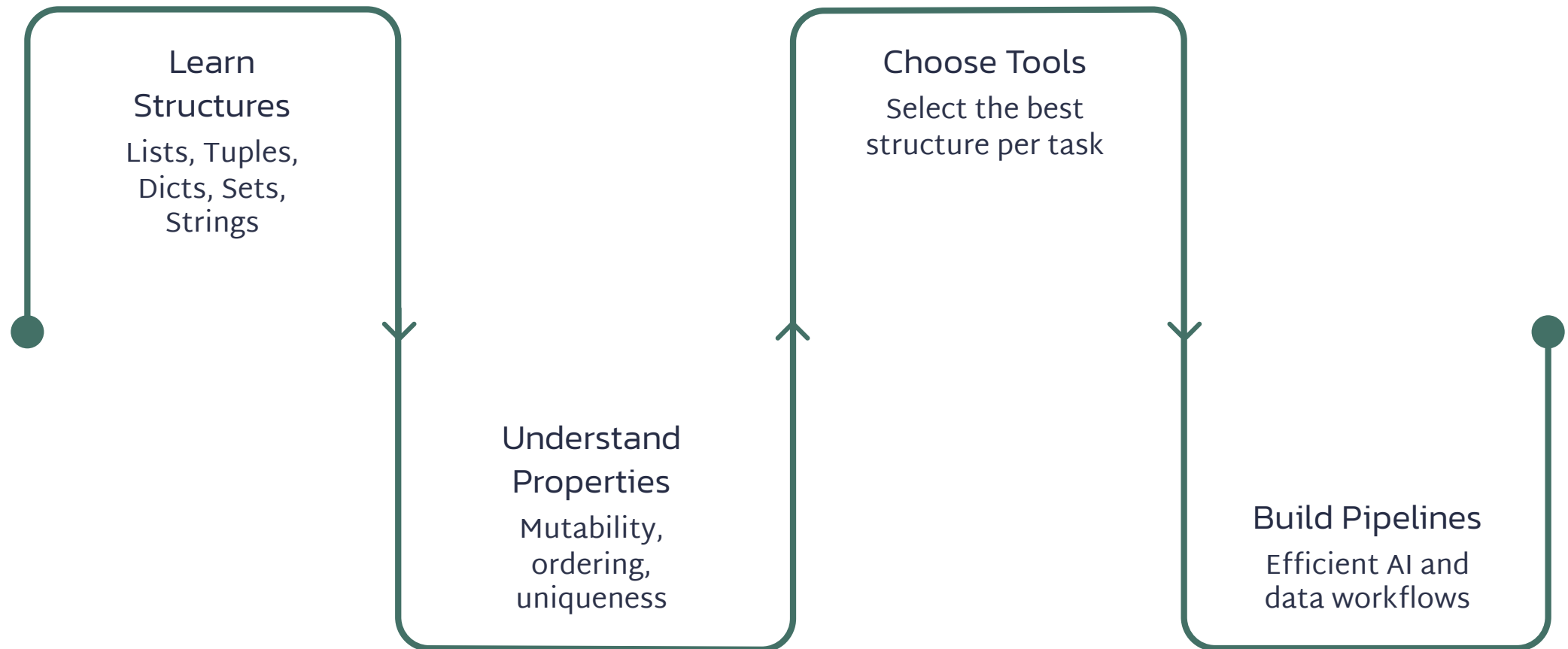
Choosing the Right Data Structure

Selecting the appropriate data structure is a fundamental skill. Use this comparison to quickly identify which structure fits your use case.

Feature	List	Tuple	Dict	Set	String
Ordered	✓ Yes	✓ Yes	✓ Yes*	✗ No	✓ Yes
Mutable	✓ Yes	✗ No	✓ Yes	✓ Yes	✗ No
Duplicates	✓ Allowed	✓ Allowed	Keys: No	✗ No	✓ Allowed
Indexed	✓ By position	✓ By position	✓ By key	✗ No	✓ By position
Best For	Sequences	Fixed records	Lookups	Uniqueness	Text data

* Dictionaries maintain insertion order since Python 3.7+

Key Takeaways



Mastering Python's built-in data structures is **the foundation** for everything in AI and data science — from cleaning raw datasets to building complex model pipelines. Practice using each structure in real scenarios to build intuition for when to reach for which tool.

"Bad programmers worry about the code. Good programmers worry about data structures and their relationships." — Linus Torvalds